

Executive Summary

Issue: SSO fails in the non-prod/test environment for the Croesus SaaS (a vendor-hosted AWS application) due to an **Azure AD Conditional Access** conflict. In production, user sign-ins succeed because devices are Intune-compliant in the production tenant, satisfying strict Conditional Access (CA) policies. In the test (non-prod) Microsoft Entra ID tenant, however, **Croesus's backend performs an OAuth On-Behalf-Of (OBO) token exchange** from an **external AWS IP**, causing a second Azure AD token issuance. This second request lacks the original user device context and originates from outside the corporate network. **Conditional Access re-evaluates the sign-in and fails it** because the device isn't marked *compliant* in the test tenant (a device can only be compliant in one tenant) and the new IP is untrusted ¹. The result: a design mismatch between **(1)** separate identity environments (prod vs. test), **(2)** strict device-based CA, and **(3)** the SaaS's backend token reuse approach.

Solution Spectrum: We outline **three architecture options** to address this problem (see **Table 1** below for high-level comparison):

- **Option 1: Adjust the Conditional Access Policies for Non-Prod** – e.g. **introduce exceptions** or relax certain controls for the test environment to accommodate the SaaS's OBO flow.
- **Option 2: Leverage cross-tenant trust or identity integration** – e.g. **use Azure AD B2B** or cross-tenant access settings to trust device compliance claims from the production tenant ², reducing policy friction in the test tenant without weakening security globally.
- **Option 3: Unify or re-design the identity strategy** – e.g. **use a single Entra tenant for both prod and test** (isolating test at the app level or via separate user groups), or other identity architecture changes that **remove the cross-tenant compliance gap** altogether.

Each approach involves trade-offs between **security posture, usability, complexity, and cost**. We detail these options with **pros, cons, risks, and mitigations**, then provide a breakdown of **relevant OAuth 2.0 / OpenID Connect flows** (Authorization Code + PKCE and OBO) from a Microsoft Entra perspective. Finally, we give **operational guidance** on analyzing Azure AD

sign-in logs (to differentiate the two-step authentication behavior and to support discussions with security teams).

Non-Prod SSO Failure: Root Cause

Azure AD Conditional Access re-check fails the SaaS's *backend token reuse* step in test environment because the user's device isn't *marked compliant* in the test tenant, and the second token request comes from an **AWS** IP outside the trust perimeter, violating device-based policy.

Design Challenge

The organization must balance three conflicting requirements: **strict device compliance policies, prod vs non-prod environment separation**, and the SaaS's **OBO-style token usage**. These conflict under the current architecture, prompting a need for design changes or exceptions.

Architecture Options for Addressing the SSO Issue

Below we detail three **feasible architecture options** to resolve the non-prod SSO failure, assuming no immediate changes to the Croesus SaaS implementation. **Each option addresses the identity/CA mismatch in a different way**, and is evaluated through the lens of security, usability, complexity, cost, **risks**, and **potential mitigations**:

Option 1: Modify Conditional Access Policies for Non-Prod (Allow Exceptions)

Description: Adjust the test tenant's Conditional Access rules to **allow the OBO flow** to succeed. This could include **exempting the Croesus non-prod application or its backend from certain requirements** or **defining a special CA policy** with less restrictive controls for a limited set of test users and resources. For example, the test tenant might not strictly require a compliant device for the specific non-prod SaaS app or could designate the SaaS's backend IP addresses as "trusted locations" in CA.

- **Security:** *Weakened if not carefully scoped.* An exception to device compliance or location policy **increases risk** by creating a potential gap in

Zero Trust controls. Attackers might attempt to exploit the exception if not tightly limited.

- **Usability:** *High*. Eliminating the compliance requirement (or trusting the AWS IP range) would likely **resolve SSO failures** immediately for testers, allowing seamless non-prod access similar to production (no extra prompts, no special infrastructure needed).
- **Operational Complexity:** *Low to Medium*. Changing CA policies is straightforward through the Entra admin portal. However, **precise scoping** is critical – e.g. apply the relaxed policy only to a specific user group and specific app, or only to the test tenant. Admins must monitor for any misuse of this exception.
- **Costs:** *Minimal direct cost*. It mostly involves administrative effort. There's no new infrastructure, but **there may be “hidden” security costs** if the exception leads to additional risk mitigation overhead or potential incidents.
- **Risks & Mitigations:** This approach **reduces security** by punching holes in your CA posture. For mitigation, **limit the scope**: apply the relaxed policy only to *non-production testers and the test app*, and consider **time-bound or conditional** exceptions. Monitor sign-in logs for any anomalies beyond expected test usage. If using *trusted IP addresses*, acknowledge that AWS IP ranges can change; restrict by region if possible and require MFA as a compensating control (so an attacker from an AWS IP would still need MFA). Log and review any sign-ins that use the exception to quickly detect abuse. Overall, ensure leadership and security teams accept the trade-off since this option explicitly trades some security strictness for immediate usability.

Option 2: Cross-Tenant Device Compliance Trust (Leverage Non-Prod Tenant Differently)

Description: Preserve the separate non-prod Entra tenant **but integrate it via cross-tenant access** so that it *trusts* the production tenant's security claims. Specifically, you can use Microsoft Entra **B2B collaboration**: invite the production users as guest accounts in the test tenant (or use **Cross-Tenant Access Settings** for B2B) and **enable “Trust compliant devices”** for the partner (prod) organization ². In this model, testers log into the Croesus test environment with their *main production credentials* via B2B. The test tenant's

Conditional Access will accept the device compliance state from the home tenant if configured.

- **Security:** *Maintained or Moderately High.* This option **avoids weakening CA** for everyone. Device compliance is still required (no blanket exceptions), preserving Zero Trust principles. The *new trust relationship* does increase complexity (two tenants trust each other's claims), but the trust can be limited to specific purposes (e.g. only trust compliance from the one partner tenant).
- **Usability:** *Medium.* This approach addresses the login failures without requiring end-users to use special accounts or bypass security. From the testers' perspective, they continue using their usual credentials, but might notice a slightly different login experience (B2B redemption or a switch of context). The friction is minimal once configured – no additional prompts beyond possibly an initial acceptance to join as a guest.
- **Operational Complexity:** *Medium to High.* **Implementing cross-tenant device trust** requires **one-time configuration** in both the production and test tenants (Azure AD External Identities settings). It's more complex than simply toggling a CA policy. Ongoing maintenance is minimal once set up, but careful configuration is required (e.g., ensuring test tenant policies properly trust incoming device claims from the prod tenant). **User management** becomes an extra step: test users must be invited or synced as guests.
- **Costs:** *Moderate.* There's no additional licensing cost for basic cross-tenant collaboration (it's built into Entra ID B2B). However, it may require some **engineering and testing effort** to validate that compliance claims are flowing as expected. If external collaboration features are new in your org, it may involve internal alignment (less quantifiable).
- **Risks & Mitigations:** Cross-tenant trust introduces an **expanded trust boundary**. The test tenant is now implicitly relying on the security posture of the production tenant for certain signals (like device compliance). A misconfiguration or compromise in one tenant could impact the other. Mitigate by **limiting trust to specific claims** (e.g. trust only device compliance and/or MFA, not all claims) and **monitor cross-tenant signs**. Ensure residual risk is acceptable — for example, that test tenant holds only non-sensitive data since it's integrated with external identities. Also, test thoroughly: cross-tenant device compliance is a relatively

advanced feature, so verify it covers this scenario (Azure AD does support trusting device claims for external users to satisfy CA [2](#)). If done properly, this solution offers strong security with manageable risk.

Option 3: Unify or Redesign the Identity Architecture (Single Tenant or Hybrid Approach)

Description: Reconsider the strict separation of prod vs. non-prod at the identity layer. **Using one Azure AD tenant for both environments** (with separate application instances or separate user groups for testing) would **eliminate cross-tenant device compliance issues** entirely – the user's device would be recognized as managed/compliant in all contexts. Alternatively, if a separate non-prod tenant is mandatory (e.g. for vendor contractual reasons), you could explore using **dedicated test devices** that are enrolled in the test tenant, or an Azure AD Hybrid configuration where a single identity is federated to multiple tenants (complex). The simplest of these is a single tenant with **logical separation** of prod vs. test application instances.

- **Security:** *Very High (if unified), or High (if hybrid).* A single-tenant model retains full CA enforcement on all sign-ins (no exceptions needed), maintaining a robust security posture. The main risk is that combining environments might break the intended isolation between prod and test – if not managed carefully, a breach or misconfig in test could potentially affect production. But identity-level unification *can be done while still isolating actual application data* by using separate app registrations or multi-instance capabilities.
- **Usability:** *High (for unified tenant).* Users seamlessly access both prod and non-prod with the same credentials and from the same devices without jumping through extra hoops. Backend token flows (OBO) work identically in test as in production because Azure AD sees the same managed device and user identity context each time. If instead using dedicated test devices joined to the test tenant, usability is *lower*, as testers must use separate machines or profiles for non-prod.
- **Operational Complexity:** *Varies.* If moving to a single tenant, there is a **one-time project** to consolidate (migrating the test SSO configuration from the separate tenant into the prod tenant as a separate app instance or environment). This requires planning and coordination (especially if previously relying on separate directories for isolation). If going the route

of dedicated test devices, the organization would need to **manage a parallel Intune environment** for those devices – adding overhead and not scaling well.

- **Costs:** *Low to Moderate*. A unified tenant approach may even reduce costs (fewer duplicate directory licenses, simpler support). However, an **identity migration** is a project that consumes time/resources. Alternatively, equipping separate test devices (or virtual desktops) might incur hardware or licensing costs.
- **Risks & Mitigations:** Merging identity domains means **losing the “hard” separation** between prod and test identity contexts. If the test environment is for third-party integration and carries less sensitive data, this may be acceptable. Mitigate by relying on **strict role-based access control**: e.g. ensure test apps are accessible only to test user groups, limiting impact even if test gets compromised. Strong change management is needed when altering identity architecture; ensure the **security team approves** and that logging/monitoring covers both environments. In short, this option maximizes security and user simplicity but at the cost of **organizational complexity** (governance and migration effort).

Table 1: Comparison of Architecture Options

Option	Security Posture	Usability	Complexity & Operations	Direct Cost
1. Relax CA in Non-Prod (Scoped policy exceptions for test environment)	Low Security – introduces exceptions in Zero Trust controls, creating potential risk hotspots if misused or not tightly scoped.	High Usability – testers can authenticate seamlessly in non-prod (no special steps or extra prompts).	Low Complexity – relatively simple CA configuration changes; requires careful scoping and ongoing monitoring of test sign-ins.	Minimal cost – primarily administrative effort; no new infrastructure or licensing required.

Option	Security Posture	Usability	Complexity & Operations	Direct Cost
2. Cross-Tenant Device Trust (Test tenant trusts prod's compliance & MFA claims via B2B)	High Security – retains strong CA enforcement by requiring compliance, while bridging the multi-tenant gap ² . Some added trust complexity but no blanket policy bypass.	Medium-High Usability – near-transparent for users (slight initial B2B onboarding) ; no separate test accounts or devices needed.	Medium Complexity – one-time cross-tenant access configuration; maintain guest user accounts or group. More setup than Option 1 but lower long-term risk.	Moderate cost – no new licenses; some one-time implementation and testing effort to configure cross-tenant settings and potentially adjust user onboarding.

Option	Security Posture	Usability	Complexity & Operations	Direct Cost
3. Unified/Hybrid Identity (Use single Entra tenant for both prod & test, or manage test devices in test tenant)	Very High Security (if single tenant) – no CA exceptions; consistent policies across environments. Slightly reduced if separate test devices route (since testers might use less familiar devices).	High Usability (for single tenant) – one credential & device covers all. If separate test devices: lower user convenience (managing multiple devices/contexts).	High Complexity – requires strategic changes (e.g. merging tenants or adding test device MGMT). Increased governance and planning needed; once done, simpler day-to-day.	Moderate cost – potential cost savings if eliminating a tenant. But initial investment for migration or test device setup is needed.

OAuth 2.0 / OpenID Connect Flow Explanation (Microsoft Entra ID Context)

To clarify the technical underpinnings, this section explains how a **standard Azure AD user login and token flow** works in this scenario and how the **SaaS's backend token reuse (OBO)** changes the context and triggers Conditional Access differently. **Figure 1** below illustrates the two phases of authentication and highlights where Conditional Access policies are applied:

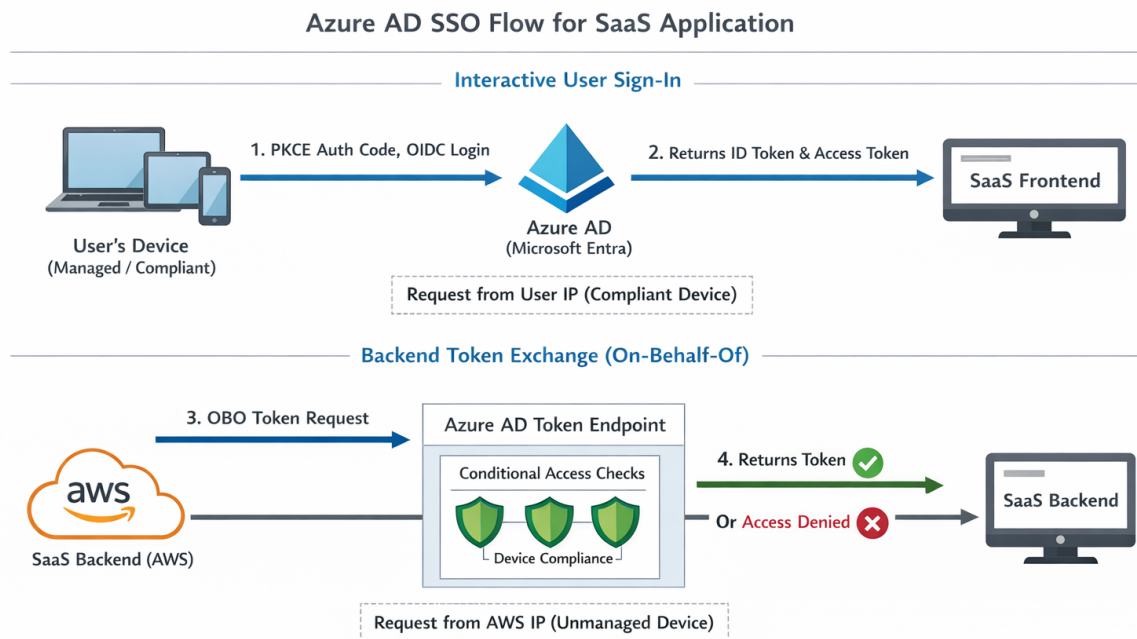


Figure 1: Split authentication flow for Croesus SaaS: initial user sign-in vs. subsequent on-behalf-of token request, showing where Conditional Access checks device compliance and sees different IP & device context.

- Initial User Sign-In (Auth Code + PKCE, OIDC):** In a typical Azure AD **OpenID Connect** sign-in sequence, the end user on a managed corporate device is redirected to **Azure AD** for authentication. The **Authorization Code + PKCE** flow (Proof Key for Code Exchange) is the recommended modern approach: the user's device (e.g., a compliant Windows PC) obtains an **authorization code** from Azure AD after successful interactive login (with multi-factor authentication if required), then exchanges that code for **tokens**. Azure AD returns an **ID token (OpenID Connect)** to authenticate the user to the app, and often an **access token (OAuth2)** for the app's backend service, plus a refresh token for session continuity. **Conditional Access is enforced at token issuance time** – e.g., requiring MFA or a compliant device if policy mandates ¹. In this first step, the **user's device state and IP are evaluated by CA**. In production, the device is marked "compliant" in Azure AD/Intune, and likely on a trusted network, so the initial token issuance satisfies all CA conditions. In a test tenant where device compliance isn't present, the **initial sign-in might still succeed** if the CA policy is less strict for that app or user (or if CA conditions only

apply to certain apps or token types; not all CA policies necessarily applied at this stage).

- **Backend Token Exchange (OAuth 2.0 On-Behalf-Of Flow):** After the user is signed in, the Croesus **SaaS backend** (running on AWS) attempts to use the user's token to request another token – for example, an access token to call another API or service, or even to validate the user's session (details depend on Croesus's design). This is an **On-Behalf-Of (OBO)** flow: the SaaS's backend (a confidential client) presents the user's initial access token (or a special token) to Azure AD's token endpoint, requesting a **new access token for a downstream resource** while asserting the user's identity ¹. At this stage, **Azure AD performs Conditional Access checks again**, but now the “sign-in” appears to come from the **SaaS backend's environment (AWS)**, *not* the user's device. The user's original device compliance claim typically **does not transfer** to this new request (and certainly not if it's a different tenant) ¹. Therefore:
 - **In production:** The second token issuance still passes if the device is known to Azure AD and the CA policy is satisfied (for example, if device compliance is recognized and external IPs are allowed due to compliance).
 - **In non-prod (test) environment:** **Azure AD sees a request from an untrusted IP (the vendor's AWS servers) and an unrecognized device context.** If a CA policy requires a compliant or domain-joined device, the request **fails** because the user's device isn't registered/compliant in that tenant, and the seen IP is outside the corporate network. Azure AD thus blocks the token issuance (as shown by the red “Access Denied” path in Figure 1). This is a known limitation: **OBO flows cannot satisfy device-based Conditional Access policies** or other “step-up” requirements without user interaction ¹. The user is left with a failure in the SSO sequence after initial login.

Conditional Access Evaluation Points: *In summary, Azure AD enforces CA at each token issuance event.* The interactive user login triggers CA checks with the user's device & location context. The OBO token request triggers a **new CA evaluation** (now with the SaaS backend as the requesting client). This second evaluation sees different conditions (IP address and device state) leading to the policy violation. The fundamental challenge is that **the user's corporate**

device signals (compliance, location) that satisfied the first login do not carry over to the second hop initiated by the cloud backend.

Operator Guidance: Analyzing Azure AD Sign-In Logs & Explaining to Security Teams

To support a data-driven discussion with security architects, operators can leverage **Microsoft Entra sign-in logs** to illustrate the two-phased authentication and pinpoint the Conditional Access failure:

- **Identify both sign-in events:** In Azure AD's sign-in logs, you will find the user's **initial interactive sign-in** (Type: *interactive user sign-in*) followed by a **second sign-in event** for the OBO token request (Type: *non-interactive user sign-in*, since it involved a service requesting a token on behalf of the user). Filter logs by the user and timestamp of an SSO attempt to correlate these events.
- **Compare Client App and Resource fields:** The first event will show the **client application** as the user's browser or user-agent (e.g. "Browser" or the name of the front-end app). The **target resource** might be the Croesus app's App ID or an associated API. The second event often shows a **different client** (the confidential app representing Croesus's backend) and possibly a different **resource** (if the backend was calling an external API like Microsoft Graph or a separate app ID). Recognizing this pattern helps confirm an OBO flow.
- **Note IP addresses and locations:** The initial sign-in will log the **user's actual IP** (e.g., an office or home IP address in Canada). The subsequent OBO-related sign-in will typically log an **AWS cloud IP address** (likely a different region/country than the user's location). This "impossible travel" gap – a near-simultaneous sign-in from a distant location – can even trigger an **Azure AD Identity Protection "Atypical travel" risk alert** if risk policies are enabled. **Explaining this to security:** Emphasize that the second sign-in is not a malicious login by the user, but **the SaaS performing a server-side token request**, making it appear as if the user's session moved to an AWS data center.
- **Check device information and CA status:** The log for the second failed sign-in will show that **Conditional Access policies blocked the attempt**, with details indicating the **unmet requirement** (e.g., "Require device to be

marked as compliant” or a location-based policy) and that the device was “**Not compliant**” (or “Unknown”) in that tenant. Compare this with the first sign-in event where the **device state** for the user likely shows as compliant (if it’s in the prod tenant). This contrast in device compliance state between the two events is key evidence.

- **Correlate with Conditional Access report:** Use the “**Report-only**” mode or **What-If tool** if available to simulate the scenario. This helps demonstrate exactly which CA policy triggered the block. In discussion, point out that the second token request is effectively a **new sign-in** from a different context – illustrating why the CA engine blocked it by design.

Explaining to Security Teams: Frame the discussion around **Zero Trust principles** – the second request fails because Azure AD is doing the right thing (not trusting tokens used from untrusted contexts automatically). Present the data: show the two log entries side by side, highlighting the differences in “**Client App,**” “**Resource,**” “**Source IP,**” “**Device compliance state,**” and **Conditional Access outcome**. Security architects will recognize these signals – the goal is to confirm that the CA policy operated as configured. From there, you can discuss the aforementioned architecture options. Emphasize how each option addresses the problem: *Option 1* introduces controlled exceptions (with risk trade-offs), *Option 2* uses **Microsoft Entra B2B trust** to keep security intact by honoring compliance from the home tenant [2](#), and *Option 3* realigns the identity structure to eliminate the root cause. By using the logs and Microsoft’s own documentation, you can substantiate the recommended approach and ensure any **policy changes or design decisions are well-informed by Microsoft Entra best practices and risk considerations**.